- **College Name : Pune Vidyarthi Griha's COLLEGE OF ENGINEERING, TECHNOLOGY AND MANAGEMENT,**
- **Department Name: Department of Artificial Intelligence and Data Science**
- **Student Name & Roll No.**
- **Class & Batch: BE**

EXPERIMENT 4: Data Wrangling

Problem Statement: Data Wrangling on Real Estate Market

Prerequisites

pip install pandas numpy scikit-learn

```
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (2.2.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (2.1.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.13/dist-packages
(1.6.1)
Requirement already satisfied: python-dateutil>=2.8.2 in
/usr/local/lib/python3.13/distpackages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages
(from pandas) (2026.4)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.13/dist-packages
(from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-packages
(from scikit-learn) (1.6.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.13/distpackages (from scikit-learn) (3.7.0)
Requirement already satisfied: cloudpickle>=3.0 in
/usr/local/lib/python3.13/distpackages (from joblib>=1.2.0->scikit-learn) (3.1.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from
python-dateutil>=2.8.2->pandas) (1.17.0)
```

#Step 0: Generate Sample RealEstate_Prices.csv & Neighborhood Dataset

```
import pandas as pd import numpy as np import os

if not os.path.exists("RealEstate_Prices.csv"):
np.random.seed(42)     n = 150

   neighborhoods = ['Koregaon Park', 'Baner', 'Wakad', 'Viman Nagar', 'Kothrud', 'Hinjewadi']
prop_types = ['Apartment', 'Villa', 'Row House', 'Commercial']     furnishing = ['Unfurnished',
'Semi-Furnished', 'Fully-Furnished']
```

```
# 1. Create main Real Estate dataset with messy column names
raw_data = {
    'Property ID #': [f'PROP-{1000+i}' for i in range(n)],
    'Neighborhood / Locality': np.random.choice(neighborhoods, n),
    'Property Type!': np.random.choice(prop_types, n, p=[0.60, 0.15, 0.15, 0.10]),
    'Area (Sq.Ft.)': np.random.randint(650, 3200, n).astype(float),
    'BHK_Count': np.random.choice([1, 2, 3, 4], n, p=[0.15, 0.45, 0.30, 0.10]).astype(float),
    'Furnishing Status': np.random.choice(furnishing, n),
    'Property Age (Yrs)': np.random.randint(0, 20, n).astype(float),
    'Sale Date': pd.date_range(start='2023-01-01', periods=n, freq='7D').strftime('%Y-%m-%d'),
'Sale Price (INR Lakhs)': np.round(np.random.uniform(45.0, 280.0, n), 2)
}

df_raw = pd.DataFrame(raw_data)

# Inject missing values df_raw.loc[5:12, 'Area
(Sq.Ft.)'] = np.nan df_raw.loc[15:20,
'BHK_Count'] = np.nan df_raw.loc[25:28,
'Furnishing Status'] = np.nan
df_raw.loc[30:32, 'Sale Price (INR Lakhs)'] = np.nan # Target variable missing

# Inject extreme outliers df_raw.loc[40, 'Sale Price (INR Lakhs)'] =
1450.0 # Extreme price outlier df_raw.loc[41, 'Area (Sq.Ft.)'] = 15000.0
# Extreme area outlier

df_raw.to_csv("RealEstate_Prices.csv", index=False)

# 2. Create supplementary Neighborhood Amenities dataset for Task 3 Merging
amenities_data = {
    'Neighborhood': neighborhoods,
    'Metro_Distance_km': [2.5, 4.0, 5.5, 1.2, 0.8, 3.0],
    'Livability_Index': [9.2, 8.6, 8.1, 8.9, 8.7, 7.9],
    'Nearby_Schools_Count': [12, 15, 18, 14, 16, 10]
}
pd.DataFrame(amenities_data).to_csv("Neighborhood_Amenities.csv", index=False) print("Sample
'RealEstate_Prices.csv' and 'Neighborhood_Amenities.csv' generated successfully.\n") Sample
'RealEstate_Prices.csv' and 'Neighborhood_Amenities.csv' generated successfully.

import pandas as pd import
numpy as np
import re from sklearn.preprocessing import
LabelEncoder
```

TASK 1: Import Dataset & Clean Column Names

```
df = pd.read_csv("RealEstate_Prices.csv") print("---
Task 1: Cleaning Column Names ---")
print("Original Columns:", df.columns.tolist())
```

```
# Remove special characters, replace spaces/slashes with underscores, and strip trailing underscores
df.columns = [
    re.sub(r'_+', '_', re.sub(r'[^\w\s]', '', col).strip().replace(' ', '_'))
for col in df.columns
]



# Rename specific columns for concise, standardized clarity rename_map
= {
    'Property_ID': 'Property_ID',
    'Neighborhood_Locality': 'Neighborhood',
    'Property_Type': 'Property_Type',
    'Area_SqFt': 'Area_SqFt',
    'BHK_Count': 'BHK',
    'Furnishing_Status': 'Furnishing_Status',
    'Property_Age_Yrs': 'Property_Age_Yrs',
    'Sale_Date': 'Sale_Date',
    'Sale_Price_INR_Lakhs': 'Sale_Price_Lakhs'
}
df.rename(columns=rename_map, inplace=True) print("Cleaned
Columns :", df.columns.tolist())
--- Task 1: Cleaning Column Names ---
Original Columns: ['Property ID #', 'Neighborhood / Locality', 'Property Type!', 'Area
(Sq.Ft.)', 'BHK_Count', 'Furnishing Status', 'Property Age (Yrs)', 'Sale Date', 'Sale
Price (INR Lakhs)']
Cleaned Columns : ['Property_ID', 'Neighborhood', 'Property_Type', 'Area_SqFt', 'BHK',
'Furnishing_Status', 'Property_Age_Yrs', 'Sale_Date', 'Sale_Price_Lakhs']
```

TASK 2: Handle Missing Values

```
print("\n--- Task 2: Handling Missing Values ---")
print("Missing Values Before Cleaning:\n", df.isnull().sum()[df.isnull().sum() > 0])

# 1. Removal Strategy: Drop rows where the target variable ('Sale_Price_Lakhs') is missing
df.dropna(subset=['Sale_Price_Lakhs'], inplace=True)

# 2. Group-Based Median Imputation: Fill missing 'Area_SqFt' using median area per Property_Type
df['Area_SqFt'] = df.groupby('Property_Type')['Area_SqFt'].transform(    lambda x:
x.fillna(x.median())
)

# 3. Median Imputation: Fill remaining numeric columns (e.g., BHK, Property_Age_Yrs) with median
num_cols = df.select_dtypes(include=[np.number]).columns for col in num_cols:          if
df[col].isnull().sum() > 0:
    df[col] = df[col].fillna(df[col].median())
```

```
# 4. Mode Imputation: Fill categorical missing values with the most frequent category
cat_cols = df.select_dtypes(include='object').columns for col in cat_cols: if
df[col].isnull().sum() > 0:
    df[col] = df[col].fillna(df[col].mode()[0])

print("Total Missing Values After Cleaning:", df.isnull().sum().sum())

--- Task 2: Handling Missing Values --- Missing
Values Before Cleaning:
 Area_SqFt             8 BHK
6
Furnishing_Status     4 Sale_Price_Lakhs
3
dtype: int64
Total Missing Values After Cleaning: 0
```

TASK 3: Data Merging with External Dataset

```
print("\n--- Task 3: Merging Neighborhood Amenities Data ---") amenities_df
= pd.read_csv("Neighborhood_Amenities.csv")

# Perform a left join on 'Neighborhood' to enrich property records with local infrastructure metrics
df_merged = pd.merge(df, amenities_df, on='Neighborhood', how='left') print(f"Shape
before merge: {df.shape} -> Shape after merge: {df_merged.shape}")
print(df_merged[['Property_ID', 'Neighborhood', 'Sale_Price_Lakhs', 'Metro_Distance_km',
'Livability_Index']].head(3))

--- Task 3: Merging Neighborhood Amenities Data ---
Shape before merge: (147, 9) -> Shape after merge: (147, 12)
  Property_ID Neighborhood  Sale_Price_Lakhs  Metro_Distance_km  \
0   PROP-1000  Viman Nagar            133.58                1.2
1   PROP-1001      Kothrud            153.73                0.8    2   PROP-
    1002        Wakad            115.82                5.5

   Livability_Index
0               8.9
1               8.7
2               8.1
```

TASK 4: Filter and Subset the Data

```
print("\n--- Task 4: Filtering and Subsetting ---")
df_merged['Sale_Date'] = pd.to_datetime(df_merged['Sale_Date'])
residential_types = ['Apartment', 'Villa', 'Row House'] df_filtered
= df_merged[
  (df_merged['Property_Type'].isin(residential_types)) &
  (df_merged['Sale_Date'] >= '2024-01-01') &
  (df_merged['Area_SqFt'] >= 600)
].copy().reset_index(drop=True)

print(f"Rows before filtering: {len(df_merged)} | Rows after residential & date filtering:
{len(df_filtered)}")
```

```
--- Task 4: Filtering and Subsetting ---
Rows before filtering: 147 | Rows after residential & date filtering: 83
```

TASK 7: Identify and Handle Outliers

```
print("\n--- Task 7: Identifying & Handling Outliers (IQR Method) ---") outlier_cols
= ['Area_SqFt', 'Sale_Price_Lakhs']

for col in outlier_cols:
   Q1 = df_filtered[col].quantile(0.25)
   Q3 = df_filtered[col].quantile(0.75)
   IQR = Q3 - Q1     lower_bound
= Q1 - 1.5 * IQR     upper_bound
= Q3 + 1.5 * IQR

   outliers = ((df_filtered[col] < lower_bound) | (df_filtered[col] > upper_bound)).sum()
print(f"{col}: Detected {outliers} outliers outside [{lower_bound:.2f}, {upper_bound:.2f}]")

   # Cap/clip extreme values to the upper and lower IQR fences
   df_filtered[col] = df_filtered[col].clip(lower=lower_bound, upper=upper_bound)

# Derive Price_Per_SqFt (in INR) after outlier treatment
df_filtered['Price_Per_SqFt_INR'] = ((df_filtered['Sale_Price_Lakhs'] * 100000) /
df_filtered['Area_SqFt']).round(2)
```

```
--- Task 7: Identifying & Handling Outliers (IQR Method) ---
Area_SqFt: Detected 0 outliers outside [-112.25, 3781.75]
Sale_Price_Lakhs: Detected 0 outliers outside [-73.28, 366.16]
```

TASK 6: Aggregate Data for Summary Statistics

```
print("\n--- Task 6: Data Aggregation ---")

# 1. Aggregation by Neighborhood
neighborhood_summary = df_filtered.groupby('Neighborhood').agg(
   Total_Listings=('Property_ID', 'count'),
   Avg_Sale_Price_Lakhs=('Sale_Price_Lakhs', 'mean'),
   Median_Sale_Price_Lakhs=('Sale_Price_Lakhs', 'median'),
   Avg_Price_Per_SqFt_INR=('Price_Per_SqFt_INR', 'mean'),
   Avg_Livability=('Livability_Index', 'first')
).round(2).reset_index().sort_values(by='Avg_Sale_Price_Lakhs', ascending=False)

print("\n1. Summary Statistics by Neighborhood:")
print(neighborhood_summary.to_string(index=False))

# 2. Pivot Table: Average Sale Price (Lakhs) by Neighborhood and Property Type
price_pivot = df_filtered.pivot_table(    values='Sale_Price_Lakhs',
index='Neighborhood',    columns='Property_Type',    aggfunc='mean'
).round(2)
```

```
print("\n2. Average Sale Price (₹ Lakhs) Pivot Table (Neighborhood vs. Property Type):")
print(price_pivot)
```

```
--- Task 6: Data Aggregation ---

1. Summary Statistics by Neighborhood:
 Neighborhood  Total_Listings  Avg_Sale_Price_Lakhs  Median_Sale_Price_Lakhs
Avg_Price_Per_SqFt_INR  Avg_Livability
        Baner              16                161.70                   159.12
11096.47             8.6
    Hinjewadi              12                154.43                   144.76
10714.23             7.9
        Wakad              11                153.66                   163.02
7035.90              8.1
      Kothrud              13                151.60                   168.23
9074.79              8.7
Koregaon Park              14                148.95                   148.50
11086.42             9.2
  Viman Nagar              17                147.29                   143.45
7665.64              8.9



2. Average Sale Price (₹ Lakhs) Pivot Table (Neighborhood vs. Property Type):
Property_Type  Apartment  Row House   Villa
Neighborhood
Baner             159.93     206.92  143.51
Hinjewadi         141.13     261.40  180.43
Koregaon Park     154.51     160.84   51.37
Kothrud           158.30      87.90  176.22
Viman Nagar       116.87     194.02  221.16
Wakad             160.70     198.10   84.62
```

TASK 5: Handle Categorical Variables (Encoding)

```
print("\n--- Task 5: Encoding Categorical Variables ---") df_encoded
= df_filtered.copy()

# 1. Ordinal / Label Encoding for ordered categories ('Furnishing_Status') furnishing_order
= {'Unfurnished': 0, 'Semi-Furnished': 1, 'Fully-Furnished': 2}
df_encoded['Furnishing_Encoded'] = df_encoded['Furnishing_Status'].map(furnishing_order)

# Label Encode 'Property_Type' for tree-based models le
= LabelEncoder()
df_encoded['Property_Type_Label'] = le.fit_transform(df_encoded['Property_Type'])

# 2. One-Hot Encoding for nominal categories ('Neighborhood' and 'Property_Type')
df_encoded = pd.get_dummies(
  df_encoded,
```

```
   columns=['Neighborhood', 'Property_Type'],
prefix=['Loc', 'Type'],    drop_first=True,
dtype=int
)

print("Encoded DataFrame Columns:") print(df_encoded.columns.tolist())

# Export final wrangled dataset
df_encoded.to_csv("Wrangled_RealEstate_Prices.csv", index=False) print("\nFinal
wrangled dataset saved to 'Wrangled_RealEstate_Prices.csv'.")
```

```
--- Task 5: Encoding Categorical Variables --- Encoded
DataFrame Columns:
['Property_ID', 'Area_SqFt', 'BHK', 'Furnishing_Status', 'Property_Age_Yrs',
'Sale_Date',
'Sale_Price_Lakhs', 'Metro_Distance_km', 'Livability_Index', 'Nearby_Schools_Count',
'Price_Per_SqFt_INR', 'Furnishing_Encoded', 'Property_Type_Label', 'Loc_Hinjewadi',
'Loc_Koregaon Park', 'Loc_Kothrud', 'Loc_Viman Nagar', 'Loc_Wakad', 'Type_Row House',
'Type_Villa']

Final wrangled dataset saved to 'Wrangled_RealEstate_Prices.csv'.
```